

Language-Driven Heterogeneous Multi-Robot Task Planning with Structured Experience Memory

Jingjing Shen*, Xiaoqi Wei*, Mitsuaki Matsumoto**, Senling Wang*, Hiroshi Takahashi*

n807043b@mails.cc.ehime-u.ac.jp, k819002y@mails.cc.ehime-u.ac.jp,

ou.shinrei.dw@ehime-u.ac.jp, takahashi@cs.ehime-u.ac.jp

*Graduate School of Science and Engineering, Ehime University, Japan

**Faculty of Science and Engineering, Ehime University, Japan

Abstract—Heterogeneous multi-robot home-service tasks require converting natural-language instructions into executable collaborative plans under capability, dependency, and execution constraints. This paper presents a language-driven task planning framework centered on a large language model (LLM) and augmented with structured experience memory. The framework explicitly models task–resource–constraint relationships, converts user instructions into a dependency-aware subtask graph, and performs team formation and task allocation under capability and parallelism constraints. In addition, decomposition results, allocation decisions, failure types, repair strategies, and execution outcomes are stored in a planning-aligned memory structure to support experience retrieval and feedback-driven update. Experiments in the AI2-THOR simulation environment with three heterogeneous robots over four home-service tasks provide preliminary evidence that the proposed design improves robustness and cross-task reuse. The framework achieves RU = 1.00 and an average GCR of 0.92, indicating high parallel utilization and strong goal coverage, while the average TCR of 0.50 suggests that strict task completion in compositional tasks remains challenging. These results indicate that structured experience memory is a useful design element for improving executability and recovery in LLM-based heterogeneous multi-robot planning.

Keywords—Multi-Robot Systems, Task Planning, Large Language Models, Experience Memory

I. INTRODUCTION

Heterogeneous multi-robot systems have strong potential in home-service scenarios, where tasks such as object organization, retrieval, cleaning, and delivery can benefit from parallel execution and functional specialization. In such settings, robots often differ in mobility, manipulation, sensing, and interaction capabilities, making coordination substantially more difficult than in homogeneous teams. Effective execution therefore requires not only task assignment, but also task decomposition, coalition formation, dependency handling, and efficient parallel scheduling under resource constraints [1].

Recent advances in large language models (LLMs) have opened a promising direction for language-driven robot planning. Prior studies have shown that LLMs can translate high-level natural-language instructions into actionable sequences [2], ground semantic instructions in robotic affordances [3], and generate executable task plans through structured prompting [4]. Moreover, current embodied AI research is moving beyond single-agent instruction following toward long-horizon planning [5], multi-agent coordination, and memory-augmented decision making [6]. Recent surveys on LLMs for multi-robot systems and memory for autonomous LLM agents also highlight coordination, retrieval,

and feedback-driven adaptation as major emerging directions [9].

Nevertheless, extending these advantages to heterogeneous multi-robot home-service scenarios remains challenging. In addition to understanding user intent, the system must coordinate multiple robots under capability constraints, resolve subtask dependencies, reduce resource conflicts, and maintain parallel efficiency under execution uncertainty. While recent embodied multi-agent planning studies [7] and SMART-LLM [8] have shown that LLMs are useful for decomposition, coalition formation, and task allocation, cross-task experience reuse and stable feedback-driven repair remain insufficiently explored in practical home-service settings.

To address this gap, this paper proposes a language-driven task planning framework for heterogeneous multi-robot home service, centered on an LLM and augmented with structured experience memory. The framework explicitly models task–resource–constraint relationships, converts natural-language instructions into a dependency-aware subtask graph, and performs team formation and task allocation under capability, dependency, and parallelism constraints. In addition, it stores decomposition results, allocation decisions, failure types, repair strategies, and execution outcomes in a planning-aligned memory structure. Through retrieval of similar prior cases and execution-feedback write-back, the framework establishes a closed loop of planning, execution, and refinement.

Experiments in the AI2-THOR home-service simulation environment with three heterogeneous robots and four representative tasks provide preliminary evidence that the proposed design improves goal coverage and strict task completion relative to no-memory, no-repair, and heuristic-allocation baselines. These results suggest that structured memory is a useful design element for improving robustness and cross-task reuse in LLM-based heterogeneous multi-robot planning.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the problem definition and the proposed framework together with the memory mechanism. Section 4 describes the experimental setup, task design, and evaluation metrics, and reports the experimental results. Section 5 concludes the paper and discusses limitations and future directions.

II. RELATED WORK

A. Planning and Task Allocation in Heterogeneous Multi-Robot Systems

Multi-robot task planning and allocation have been widely studied in heterogeneous robotic systems. Conventional methods typically formulate collaboration as a constrained

optimization or task-assignment problem, explicitly modeling capability requirements, task dependencies, and execution costs [1]. These approaches are effective in structured environments because they provide interpretable and controllable allocation strategies. However, they generally rely on pre-defined task representations and strong domain priors, which limits their ability to process open-ended natural-language instructions commonly encountered in home-service scenarios.

B. LLM-Based Embodied Task Planning

Recent advances in large language models (LLMs) have enabled language-driven robot planning by translating high-level instructions into executable plans. Prior studies have shown that LLMs can perform zero-shot task planning [2], ground language into robotic affordances [3], generate program-like task plans [4], and improve embodied reasoning through execution feedback or reasoning-acting loops [5], [6]. More recent work has extended this paradigm to multi-agent and multi-robot settings, showing the potential of LLMs for collaborative embodied planning [7], [8]. Nevertheless, most existing approaches do not explicitly address capability-aware task allocation, coalition formation, and dependency-constrained coordination in heterogeneous multi-robot systems.

C. Memory, Retrieval, and Feedback Repair for Planning Refinement

To improve the reliability and adaptability of LLM-based planning systems, recent studies have introduced memory, retrieval-augmented reasoning, and feedback-driven repair mechanisms, which reuse historical plans, execution traces, and failure cases to refine subsequent decision making [5], [6]. Such strategies are particularly valuable in long-horizon embodied tasks, where one-shot planning is often insufficient for robust execution. However, in heterogeneous multi-robot home-service scenarios, cross-task experience reuse remains underexplored, especially for task decomposition, coalition formation, allocation adjustment, and failure recovery. In contrast, our framework integrates LLM-based planning with structured experience memory, enabling retrieved prior experience and execution feedback to improve planning consistency and recovery robustness in heterogeneous multi-robot home-service tasks.

III. METHOD

A. Problem Definition

This study considers language-driven task planning for heterogeneous multi-robot home-service scenarios. Given a natural-language task instruction I , environmental information E (e.g., interactive objects and summaries of spatial layouts), and a heterogeneous robot set $\mathcal{R} = \{r_i\}_{i=1}^N$, where each robot r_i is characterized by a capability set $\mathcal{S}_i$ and a state x_i (e.g., location, battery level, and occupancy status), the goal is to generate an executable collaborative plan Π that completes the task under capability and temporal constraints while maximizing parallel execution efficiency.

We represent the collaborative plan Π as a combination of a dependency-aware subtask set $\mathcal{T} = \{t_k\}_{k=1}^K$ and an allocation function $A(\cdot)$. The subtasks form a graph $\mathcal{G} = (\mathcal{T}, \mathcal{D})$, where $(t_a \rightarrow t_b) \in \mathcal{D}$ denotes a dependency relation requiring t_a to be completed before t_b . Subtasks within the same topological layer can be executed in parallel. Each subtask t_k is assigned either to a single robot or to a robot

team $A(t_k) \subseteq \mathcal{R}$. When no individual robot can satisfy the required skills, a team is formed such that the union of their skills covers the requirement:

$$Req(t_k) \subseteq \bigcup_{r_i \in A(t_k)} \mathcal{S}_i \quad (1)$$

This skill-coverage-based formulation of team formation and allocation is consistent with prior LLM-based multi-robot planning paradigms.

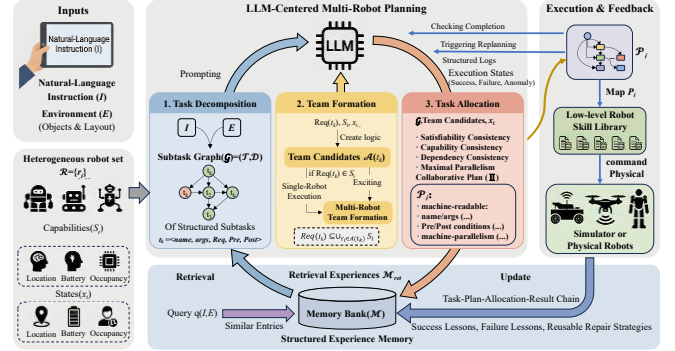


Fig. 1. Overall Framework of the Proposed Language-Driven Multi-Robot Task Planning System

B. Overall System Framework

As illustrated in Fig. 1, we propose an LLM-centered multi-robot task planning system augmented with an experience memory mechanism. The system takes as input robot types, robot counts, capability descriptions, environmental information, and task instructions. During planning, the LLM receives necessary priors and constraint conditions and outputs structured planning results. The system comprises six stages: task decomposition, team formation, task allocation, task execution, execution feedback, and memory update, thereby establishing a closed loop of planning, execution, and learning.

For reproducibility, the input to each planning cycle is defined as $(I, E, \mathcal{R}, \mathcal{S}_i, x_i)$ together with retrieved memory $\mathcal{M}_{ret}$, when available. The output consists of a subtask graph $\mathcal{G}$, team and allocation decisions $A(\cdot)$, and a structured sequence of actions or skill invocations that can be parsed by the executor.

C. Task Decomposition

In the task decomposition stage, the system converts the high-level natural-language instruction I into an executable subtask set $\mathcal{T}$ and dependency relations $\mathcal{D}$. We employ structured prompts to induce the LLM to output a unified representation consisting of subtasks, preconditions, target objects, required skills, and dependencies, thereby reducing the influence of linguistic ambiguity on subsequent team formation and allocation. In practice, each subtask t_k is represented as

$$t_k = \langle name, args, Req(t_k), Pre(t_k), Post(t_k) \rangle,$$

where $Req(t_k)$ denotes the required skill set, such as navigation, grasping, or switching operations, while $Pre(t_k)$ and $Post(t_k)$ denote key pre- and post-conditions. The decomposed subtasks are organized into a directed acyclic graph (DAG) according to their dependencies to support parallel execution.

D. Team Formation

In the team formation stage, the system constructs a set of candidate teams according to subtask requirements $Req(t_k)$, robot capabilities $\mathcal{S}_i$, and current robot states x_i , such as location, idleness, and load. For each subtask t_k , if there exists a robot r_i such that $Req(t_k) \subseteq \mathcal{S}_i$, single-robot execution is preferred. Otherwise, multiple robots are selected to form a team whose combined skills cover the requirement. This principle of prioritizing single-robot execution and resorting to cooperation only when necessary is consistent with coalition formation settings in prior LLM-based multi-robot planning studies.

E. Task Allocation

In the task allocation stage, the LLM takes as input the subtask graph $\mathcal{G}$, candidate team information, and robot states, and outputs the final allocation $A(t_k)$ together with the execution order and parallel relations. This stage must satisfy three principles: capability consistency, dependency consistency, and maximal parallelism. Specifically, capability consistency requires satisfying (1), while dependency consistency requires that if $(t_a \rightarrow t_b) \in \mathcal{D}$, then t_b starts only after t_a is completed. Maximal parallelism means that, without violating dependency or resource-conflict constraints, different robots should execute different subtasks concurrently whenever possible. The resulting allocation is organized into a machine-readable structure, such as a list of parallel subtasks at each temporal layer and their corresponding robot assignments, and is then dispatched to either a simulator or physical robots by the executor.

IV. EXPERIMENTS

A. Experimental Setup

We evaluate the proposed framework in the indoor interactive simulation platform AI2-THOR. The planner and executor are provided with symbolic world states extracted from simulator metadata, including object categories, room locations, interactive attributes, key object states, robot poses, object-holding status, and execution feedback. A unified low-level skill library supports navigation, pick-and-place, open/close or on/off interaction, and cleaning operations. The high-level planner outputs structured plans containing task decomposition, capability constraints, robot assignment, dependency relations, and parallel blocks, while the executor invokes the corresponding skills and returns failure causes to trigger online repair and memory write-back.

The planner uses GPT-4 and produces strictly formatted JSON plans via structured prompting. The temperature is set to 0.1 and the maximum token budget is 1024. The experience memory module is enabled by default: during planning, the top-3 most similar experiences are retrieved and injected into the prompt, and task summaries are written back at the end of each task or when failure occurs, supporting cross-task reuse and failure repair. Each task is repeated $N = 5$ times under different random seeds or initial perturbations. The maximum number of execution steps is set to 250, and runs exceeding this limit are regarded as time-out failures. To ensure fair comparison, all methods including the proposed approach, baselines, and ablations use the same low-level skill library and the same execution constraints. Evaluation is conducted using RU, GCR, and TCR, where GCR measures continuous goal satisfaction and TCR is a strict binary completion indicator that equals 1 only when GCR = 1.

The heterogeneous robot team includes three robots. R1 is a mobile manipulator with navigation, grasping/placing, and open/close skills, mainly used for organization, transport, and placing items into containers. R2 is a lightweight interaction robot with navigation, device-switching, and state-checking abilities, mainly used for switching devices on or off and performing final goal checking. R3 is a cleaning robot with navigation and floor-cleaning skills for kitchen or whole-house floor coverage. The task suite contains four tasks: T1, “watch TV and turn off the desk lamp and floor lamp”; T2, “put food into the refrigerator and clean the kitchen floor”; T3, “fetch a cup of water and a slice of bread”; and T4, “clean the floors of all rooms in the house.”

B. Experimental Results

Table I summarizes the overall performance of the proposed method on the four home-service tasks. The robot utilization remains $RU = 1.00$ across all tasks, indicating that the generated plans consistently involve all required robot types. In contrast, performance differences are mainly reflected in GCR and TCR, especially for multi-goal and compositional tasks.

For T1 and T4, the proposed framework achieves perfect performance, with $RU = 1.00$, $GCR = 1.00$, and $TCR = 1$, showing that the system can generate fully executable collaborative plans for both simple and large-area cleaning tasks. For T2, although all required robot types are involved, the performance drops to $GCR = 0.90$ and $TCR = 0$, suggesting that some target predicates are not fully satisfied in the final state. A similar phenomenon is observed in T3, where $GCR = 0.80$ and $TCR = 0$, indicating that the method can complete the main procedure but may still miss part of the required goals in multi-object delivery tasks. Overall, the framework achieves a mean RU of 1.00, a mean GCR of 0.92, and a mean TCR of 0.50. These results suggest that the proposed method is effective at generating highly parallel task plans with strong goal coverage, although strict task completion in compositional scenarios remains challenging.

To better understand the source of the performance gains, we compare the proposed method with three representative baselines: No-Memory, which removes experience retrieval and write-back; No-Feedback-Repair, which disables replanning and reallocation after execution failure; and Heuristic Allocation, which replaces LLM-based allocation with a rule-based strategy using capability matching and nearest-distance heuristics.

As shown in Table II, the proposed framework achieves the best overall performance, with $RU = 1.00$, $GCR = 0.92$, and $TCR = 0.50$. In comparison, No-Memory yields lower mean performance ($RU = 0.95$, $GCR = 0.85$, $TCR = 0.30$), indicating that the absence of experience reuse mainly reduces goal coverage and completion reliability. No-Feedback-Repair obtains $RU = 1.00$ and $GCR = 0.91$, but its TCR decreases to 0.30, showing that online repair is particularly important for converting partially successful executions into strict task completion. The Heuristic Allocation baseline performs worst overall, with $RU = 0.82$, $GCR = 0.80$, and $TCR = 0.25$, suggesting that rule-based allocation is less effective in maintaining both stable capability matching and high-quality parallel scheduling.

The performance gap becomes more evident on the complex tasks (T2 and T4). The proposed method reaches $GCR = 0.95$ and $TCR = 0.50$, whereas No-Memory, No-

Feedback-Repair, and Heuristic Allocation achieve only 0.80/0.20, 0.90/0.20, and 0.78/0.20, respectively. This result indicates that both structured experience memory and feedback-driven repair contribute substantially to robust execution in more demanding multi-goal scenarios.

TABLE I. MAIN RESULTS OF THE PROPOSED METHOD

Task	Instruction (short description)	RU	GCR	TCR
T1	Watch TV + turn off the desk lamp and floor lamp	1.00	1.00	1
T2	Put food into the fridge + clean the kitchen floor	1.00	0.90	0
T3	Fetch water + fetch bread	1.00	0.80	0
T4	Clean the floors of all rooms	1.00	1.00	1

TABLE II. COMPARISON WITH BASELINE METHODS

Method	RU Mean	GCR Mean	TCR Mean	GCR (Complex : T2/T4)	TCR (Complex : T2/T4)
Ours	1.00	0.92	0.50	0.95	0.50
No-Memory	0.95	0.85	0.30	0.80	0.20
No-Feedback-Repair	1.00	0.91	0.30	0.90	0.20
Heuristic Allocation	0.82	0.80	0.25	0.78	0.20

TABLE III. ABLATION STUDY ON THE MEMORY MODULE

Setting	RU Mean	GCR Mean	TCR Mean	GCR (Complex : T2/T4)	TCR (Complex : T2/T4)
top-k = 1	1.00	0.90	0.30	0.88	0.20
top-k=3 (default)	1.00	0.92	0.50	0.95	0.50
top-k = 5	1.00	0.92	0.50	0.94	0.50
Retrieval only	1.00	0.91	0.30	0.90	0.20
Retrieval + write-back	1.00	0.92	0.50	0.95	0.50
Unstructured textual memory	1.00	0.90	0.30	0.88	0.50
Structured memory (failure → repair template)	1.00	0.92	0.50	0.95	0.50

We further perform ablation studies to evaluate several key design choices in the proposed memory module, including the retrieval depth (top-k = 1, 3, 5), retrieval only versus retrieval + write-back, and structured memory versus unstructured textual memory.

As reported in Table III, a moderate retrieval depth provides the most balanced performance. Specifically, the default setting top-k = 3 achieves the best overall results, with RU = 1.00, GCR = 0.92, and TCR = 0.50, outperforming top-k = 1 and matching top-k = 5 in mean GCR while slightly improving complex-task GCR. This suggests that insufficient retrieval may limit access to useful prior experience, whereas excessive retrieval may introduce irrelevant information and increase prompt noise.

The comparison between retrieval only and retrieval + write-back further confirms the importance of continual memory updating. Enabling write-back improves GCR from 0.91 to 0.92 and TCR from 0.30 to 0.50, indicating that execution feedback contributes to more effective cross-task experience accumulation.

A similar trend is observed when comparing unstructured textual memory with structured memory. The structured

version improves mean performance from GCR/TCR = 0.90/0.30 to 0.92/0.50, and also significantly improves complex-task GCR from 0.88 to 0.95 while maintaining TCR at 0.50. These results show that memory represented in a planning-aligned structure is more effective than plain text summaries for supporting decomposition, repair, and reallocation under complex task conditions.

V. CONCLUSION

This paper presents an LLM-centered collaborative task planning framework for heterogeneous multi-robot home-service tasks. The framework transforms natural-language instructions into a dependency-aware subtask graph and performs team formation and task allocation under explicit capability constraints. In addition, through an execution-feedback-driven closed loop, the framework stores task decomposition, allocation decisions, failure types, repair strategies, and execution outcomes in a structured experience memory, enabling cross-task experience reuse and stable recovery.

Experiments in the AI2-THOR simulation environment with three heterogeneous robots on multiple home-service tasks show that the proposed method achieves better parallel utilization and task completion stability than baselines without memory, without repair, or with rule-based allocation. It also demonstrates more reliable failure recovery on complex tasks. These results indicate that structured experience memory, aligned with task decomposition and allocation variables, can reduce repeated trial-and-error and improve the executability and robustness of collaborative multi-robot planning.

ACKNOWLEDGMENT

This research was supported by TAIYO YUDEN CO., LTD. The authors would like to sincerely thank Mr. Hiromoto Nagamine, Mr. Mark Hozoji and Ms. Rie Naito of TAIYO YUDEN CO., LTD. for their valuable advice and technical support.

REFERENCES

- [1] Y. Rizk, M. Awad, and E. W. Tunstel, "Cooperative Heterogeneous Multi-Robot Systems: A Survey," *ACM Computing Surveys*, vol. 52, no. 2, pp. 1–31, 2019.
- [2] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents," in *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.
- [3] M. Ahn et al., "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," in *Proceedings of The 6th Conference on Robot Learning (CoRL)*, 2023.
- [4] I. Singh et al., "ProgPrompt: Generating Situated Robot Task Plans using Large Language Models," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [5] W. Huang et al., "Inner Monologue: Embodied Reasoning through Planning with Language Models," in *Proceedings of The 6th Conference on Robot Learning (CoRL)*, 2023.
- [6] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [7] X. Liu, X. Li, D. Guo, S. Tan, H. Liu, and F. Sun, "Embodied Multi-Agent Task Planning from Ambiguous Instruction," in *Robotics: Science and Systems (RSS)*, 2022.
- [8] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, "SMART-LLM: Smart Multi-Agent Robot Task Planning using Large Language Models," *arXiv preprint arXiv:2309.10062*, 2023.
- [9] Li, Peihan, et al. "Large language models for multi-robot systems: A survey." *arXiv preprint arXiv:2502.03814*, 2025.